With all the parentheses, the structure of the code becomes more familiar. The top line shows that describe is a method of an object named RSpec, and that the inner describe and it lines are also methods with block arguments. There's actually an important part of how this fits together that we haven't discussed (it's a method called instance_eval, which is discussed in Chapter 22, Metaprogramming, on page 373), but the basic idea is that RSpec takes the blocks that are arguments to describe and it, holds on to them, and then invokes them in order to run the spec.

You can also see that the actual expectation is also just a set of method calls. The expect method is called with an object as an argument. The result of that method is passed the to method, which itself takes an argument that is generated by calling eq. The result of the call to eq is a *matcher*, and RSpec defines a series of matchers that interact with the to method (or its sibling method not_to) to determine whether the expectation is fulfilled or not.

We'll set up our TennisScorer class, but only enough to let it satisfy this assertion:

unittesting/bdd/2/tennis_scorer.rb

```
class TennisScorer
  def score
    "0-0"
  end
end
```

Now we can run our spec again:

```
$ rspec ts_spec.rb
.***
Pending: (Failures listed here are expected and do not affect your suite's status)
  1) TennisScorer basic scoring makes the score 15-0 if the server wins a point
     # Not yet implemented
     # ./ts_spec.rb:10
  2) TennisScorer basic scoring makes the score 0-15 if the receiver wins a point
     # Not yet implemented
     # ./ts_spec.rb:11
  3) TennisScorer basic scoring makes the score 15-15 after they both win a point
     # Not yet implemented
     # ./ts_spec.rb:12
Finished in 0.00237 seconds (files took 0.05063 seconds to load)
4 examples, 0 failures, 3 pending
```

Now we have only three pending specs; the first one has been satisfied.

Let's write the next couple of specs (I've added a new one for an error case):

unittesting/bdd/3/ts_spec.rb

```
require_relative "tennis_scorer"

RSpec.describe TennisScorer do
  describe "basic scoring" do
    it "starts with a score of 0-0" do
      ts = TennisScorer.new
      expect(ts.score).to eq("0-0")
    end

    it "makes the score 15-0 if the server wins a point" do
      ts = TennisScorer.new
      ts.give_point_to(:server)
```